

Pemilik Source Code:

1. Ilham, NRP: 3125600051
2. Izyan, NRP: 3125600057

SC:

```
public class Main {
    public static void main(String[] args) {

        // Array untuk menyimpan 5 object KomidiPutar
        KomidiPutar[] daftarWahana = new KomidiPutar[5];

        // Membuat 5 object dengan data berbeda
        daftarWahana[0] = new KomidiPutar(
            "K01", "Komidi Putar 01", 10, 5000, 10);

        daftarWahana[1] = new KomidiPutar(
            "K02", "Komidi Putar 02", 12, 5000, 8);

        daftarWahana[2] = new KomidiPutar(
            "K03", "Komidi Putar 03", 15, 7000, 15);

        daftarWahana[3] = new KomidiPutar(
            "K04", "Komidi Putar 04", 10, 5000, 6);

        daftarWahana[4] = new KomidiPutar(
            "K05", "Komidi Putar 05", 20, 7000, 12);

        // Perulangan untuk memproses seluruh object
        for (int i = 0; i < daftarWahana.length; i++) {

            System.out.println("=====");

            daftarWahana[i].infoWahana();

            System.out.println("Pendapatan      : Rp"
                + daftarWahana[i].hitungPendapatan());

            daftarWahana[i].cekKapasitas();

        }
    }
}

public class KomidiPutar {
    // Atribut
    String kode;
    String nama;
    int kapasitas;
    double hargaTiket;
```

```

int jumlahPenumpang;

// Constructor
KomidiPutar(String kode, String nama, int kapasitas,
             double hargaTiket, int jumlahPenumpang) {
    this.kode = kode;
    this.nama = nama;
    this.kapasitas = kapasitas;
    this.hargaTiket = hargaTiket;
    this.jumlahPenumpang = jumlahPenumpang;
}

// Method menampilkan informasi wahana
void infoWahana() {
    System.out.println("Kode           : " + kode);
    System.out.println("Nama           : " + nama);
    System.out.println("Kapasitas        : " + kapasitas);
    System.out.println("Harga Tiket      : Rp" + hargaTiket);
    System.out.println("Jumlah Penumpang : " + jumlahPenumpang);
}

// Method menghitung pendapatan
double hitungPendapatan() {
    return hargaTiket * jumlahPenumpang;
}

// Method mengecek kapasitas
void cekKapasitas() {
    if (jumlahPenumpang >= kapasitas) {
        System.out.println("Status           : PENUH");
    } else {
        System.out.println("Status           : TERSEDIA");
    }
}
}

```

1. Jalankan

Jalankan program tanpa melakukan perubahan terlebih dahulu. **Catat hasilnya.**

```
PS C:\PENS\Tugas\Sem 3\Praktikum OOP - Java\Java\Wahana\KomidiPutar> java Main.java
=====
Kode       : K01
Nama       : Komidi Putar 01
Kapasitas  : 10
Harga Tiket : Rp5000.0
Jumlah Penumpang : 10
Pendapatan : Rp50000.0
Status     : PENUH
=====
Kode       : K02
Nama       : Komidi Putar 02
Kapasitas  : 12
Harga Tiket : Rp5000.0
Jumlah Penumpang : 8
Pendapatan : Rp40000.0
Status     : TERSEDIA
=====
Kode       : K03
Nama       : Komidi Putar 03
Kapasitas  : 15
Harga Tiket : Rp7000.0
Jumlah Penumpang : 15
Pendapatan : Rp105000.0
Status     : PENUH
=====
Kode       : K04
Nama       : Komidi Putar 04
Kapasitas  : 10
Harga Tiket : Rp5000.0
Jumlah Penumpang : 6
Pendapatan : Rp30000.0
Status     : TERSEDIA
=====
Kode       : K05
Nama       : Komidi Putar 05
Kapasitas  : 20
Harga Tiket : Rp7000.0
Jumlah Penumpang : 12
Pendapatan : Rp84000.0
Status     : TERSEDIA
```

2. Test

Buat **3 test case**, dengan ketentuan:

- 1 kondisi normal;
- 1 kondisi batas;
- 1 kondisi tidak ideal.

Contoh:

Kon disi	Input	Expec ted	Actual

Normal	Sesuai SC awal	Output normal	<pre> PS C:\PENS\Tugas\Sem 3\Praktikum OOP - Java\Java\Wahana\ ===== Kode : K01 Nama : Komidi Putar 01 Kapasitas : 10 Harga Tiket : Rp5000.0 Jumlah Penumpang : 10 Pendapatan : Rp50000.0 Status : PENUH ===== Kode : K02 Nama : Komidi Putar 02 Kapasitas : 12 Harga Tiket : Rp5000.0 Jumlah Penumpang : 8 Pendapatan : Rp40000.0 Status : TERSEDIA ===== Kode : K03 Nama : Komidi Putar 03 Kapasitas : 15 Harga Tiket : Rp7000.0 Jumlah Penumpang : 15 Pendapatan : Rp105000.0 Status : PENUH ===== Kode : K04 Nama : Komidi Putar 04 Kapasitas : 10 Harga Tiket : Rp5000.0 Jumlah Penumpang : 6 Pendapatan : Rp30000.0 Status : TERSEDIA ===== Kode : K05 Nama : Komidi Putar 05 Kapasitas : 20 Harga Tiket : Rp7000.0 Jumlah Penumpang : 12 Pendapatan : Rp84000.0 Status : TERSEDIA </pre>
--------	----------------	---------------	--

Batas	Jumlah penumpang diset ke full	Status harus penuh semua	===== Kode : K01 Nama : Komidi Putar 01 Kapasitas : 10 Harga Tiket : Rp5000.0 Jumlah Penumpang : 10 Pendapatan : Rp50000.0 Status : PENUH ===== Kode : K02 Nama : Komidi Putar 02 Kapasitas : 12 Harga Tiket : Rp5000.0 Jumlah Penumpang : 12 Pendapatan : Rp60000.0 Status : PENUH ===== Kode : K03 Nama : Komidi Putar 03 Kapasitas : 15 Harga Tiket : Rp7000.0 Jumlah Penumpang : 15 Pendapatan : Rp105000.0 Status : PENUH ===== Kode : K04 Nama : Komidi Putar 04 Kapasitas : 10 Harga Tiket : Rp5000.0 Jumlah Penumpang : 10 Pendapatan : Rp50000.0 Status : PENUH ===== Kode : K05 Nama : Komidi Putar 05 Kapasitas : 20 Harga Tiket : Rp7000.0 Jumlah Penumpang : 20 Pendapatan : Rp140000.0 Status : PENUH =====
-------	--------------------------------	--------------------------	---

Tidak ideal	Jumlah penumpang melebihi kapasitas	Harusnya status: PENUH tapi anomali, harus ada kondisi lagi yang mencegah data anomali	<pre> ===== Kode : K01 Nama : Komidi Putar 01 Kapasitas : 10 Harga Tiket : Rp5000.0 Jumlah Penumpang : 100000000 Pendapatan : Rp5.0E11 Status : PENUH ===== Kode : K02 Nama : Komidi Putar 02 Kapasitas : 12 Harga Tiket : Rp5000.0 Jumlah Penumpang : 100000000 Pendapatan : Rp5.0E10 Status : PENUH ===== Kode : K03 Nama : Komidi Putar 03 Kapasitas : 15 Harga Tiket : Rp7000.0 Jumlah Penumpang : 200000000 Pendapatan : Rp1.4E11 Status : PENUH ===== Kode : K04 Nama : Komidi Putar 04 Kapasitas : 10 Harga Tiket : Rp5000.0 Jumlah Penumpang : 200000000 Pendapatan : Rp1.0E11 Status : PENUH ===== Kode : K05 Nama : Komidi Putar 05 Kapasitas : 20 Harga Tiket : Rp7000.0 Jumlah Penumpang : 200000000 Pendapatan : Rp1.4E11 Status : PENUH </pre>
-------------	-------------------------------------	--	---

3. Temukan Ketidaksesuaian

Sekarang:

Cari minimal 1 ketidaksesuaian pada program.

Tidak harus syntax error.

Bisa berupa:

- hasil perhitungan salah;
- kondisi salah;
- data tidak diproses seluruhnya;

- output tidak sesuai kebutuhan;
- program gagal pada kondisi tertentu;
- implementasi requirement sebelumnya tidak konsisten.

Wajib dibuktikan dengan test.

tidak bisa handle kondisi di mana jumlah penumpang terlalu banyak

```

=====
Kode       : K01
Nama       : Komidi Putar 01
Kapasitas  : 10
Harga Tiket : Rp5000.0
Jumlah Penumpang : 100000000
Pendapatan : Rp5.0E11
Status     : PENUH
=====
Kode       : K02
Nama       : Komidi Putar 02
Kapasitas  : 12
Harga Tiket : Rp5000.0
Jumlah Penumpang : 10000000
Pendapatan : Rp5.0E10
Status     : PENUH
=====
Kode       : K03
Nama       : Komidi Putar 03
Kapasitas  : 15
Harga Tiket : Rp7000.0
Jumlah Penumpang : 20000000
Pendapatan : Rp1.4E11
Status     : PENUH
=====
Kode       : K04
Nama       : Komidi Putar 04
Kapasitas  : 10
Harga Tiket : Rp5000.0
Jumlah Penumpang : 20000000
Pendapatan : Rp1.0E11
Status     : PENUH
=====
Kode       : K05
Nama       : Komidi Putar 05
Kapasitas  : 20
Harga Tiket : Rp7000.0
Jumlah Penumpang : 20000000
Pendapatan : Rp1.4E11
Status     : PENUH

```

4. Final Fix

Jika menemukan masalah:

lakukan perbaikan pada source code yang diterima.

Tetapi:

Jangan rewrite seluruh program.

Perubahan harus dilakukan pada bagian yang relevan.

Sebelum:

```
void cekKapasitas() {  
    if (jumlahPenumpang >= kapasitas) {  
        System.out.println("Status          : PENUH");  
    } else {  
        System.out.println("Status          : TERSEDIA");  
    }  
}
```

Sesudah:

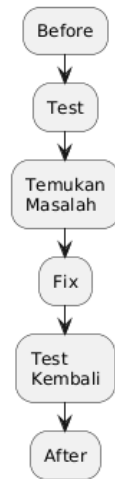
```
void cekKapasitas() {  
    if (jumlahPenumpang > kapasitas) {  
        System.out.println("Status          : MELEBIHI KAPASITAS");  
    }  
    else if (jumlahPenumpang >= kapasitas) {  
        System.out.println("Status          : PENUH");  
    } else {  
        System.out.println("Status          : TERSEDIA");  
    }  
}
```

5. Verifikasi Final

Setelah diperbaiki:

Lakukan kembali **3 test case**.

Kemudian bandingkan:



Target akhirnya:

PASS

Kon disi	Input	Expect ed	Actual

Normal	Sesuai SC awal	Output normal	<pre>PS C:\PENS\Tugas\Sem 3\Praktikum OOP - Java\Java\Wahan: ===== Kode : K01 Nama : Komidi Putar 01 Kapasitas : 10 Harga Tiket : Rp5000.0 Jumlah Penumpang : 10 Pendapatan : Rp50000.0 Status : PENUH ===== Kode : K02 Nama : Komidi Putar 02 Kapasitas : 12 Harga Tiket : Rp5000.0 Jumlah Penumpang : 8 Pendapatan : Rp40000.0 Status : TERSEDIA ===== Kode : K03 Nama : Komidi Putar 03 Kapasitas : 15 Harga Tiket : Rp7000.0 Jumlah Penumpang : 15 Pendapatan : Rp105000.0 Status : PENUH ===== Kode : K04 Nama : Komidi Putar 04 Kapasitas : 10 Harga Tiket : Rp5000.0 Jumlah Penumpang : 6 Pendapatan : Rp30000.0 Status : TERSEDIA ===== Kode : K05 Nama : Komidi Putar 05 Kapasitas : 20 Harga Tiket : Rp7000.0 Jumlah Penumpang : 12 Pendapatan : Rp84000.0 Status : TERSEDIA</pre>
--------	----------------	---------------	--

Batas	Jumlah penumpang diset ke full	Status harusnya penuh semua	===== Kode : K01 Nama : Komidi Putar 01 Kapasitas : 10 Harga Tiket : Rp5000.0 Jumlah Penumpang : 10 Pendapatan : Rp50000.0 Status : PENUH ===== Kode : K02 Nama : Komidi Putar 02 Kapasitas : 12 Harga Tiket : Rp5000.0 Jumlah Penumpang : 12 Pendapatan : Rp60000.0 Status : PENUH ===== Kode : K03 Nama : Komidi Putar 03 Kapasitas : 15 Harga Tiket : Rp7000.0 Jumlah Penumpang : 15 Pendapatan : Rp105000.0 Status : PENUH ===== Kode : K04 Nama : Komidi Putar 04 Kapasitas : 10 Harga Tiket : Rp5000.0 Jumlah Penumpang : 10 Pendapatan : Rp50000.0 Status : PENUH ===== Kode : K05 Nama : Komidi Putar 05 Kapasitas : 20 Harga Tiket : Rp7000.0 Jumlah Penumpang : 20 Pendapatan : Rp140000.0 Status : PENUH =====
-------	--------------------------------	-----------------------------	---

Tidak ideal	Jumlah penumpang diseti melebihi kapasitas	Status: MELEBIHI KAPASITAS	<pre> ===== Kode : K01 Nama : Komidi Putar 01 Kapasitas : 10 Harga Tiket : Rp5000.0 Jumlah Penumpang : 10000 Pendapatan : Rp5.0E7 Status : MELEBIHI KAPASITAS ===== Kode : K02 Nama : Komidi Putar 02 Kapasitas : 12 Harga Tiket : Rp5000.0 Jumlah Penumpang : 12000 Pendapatan : Rp6.0E7 Status : MELEBIHI KAPASITAS ===== Kode : K03 Nama : Komidi Putar 03 Kapasitas : 15 Harga Tiket : Rp7000.0 Jumlah Penumpang : 15000 Pendapatan : Rp1.05E8 Status : MELEBIHI KAPASITAS ===== Kode : K04 Nama : Komidi Putar 04 Kapasitas : 10 Harga Tiket : Rp5000.0 Jumlah Penumpang : 10000 Pendapatan : Rp5.0E7 Status : MELEBIHI KAPASITAS ===== Kode : K05 Nama : Komidi Putar 05 Kapasitas : 20 Harga Tiket : Rp7000.0 Jumlah Penumpang : 20000 Pendapatan : Rp1.4E8 Status : MELEBIHI KAPASITAS </pre>
-------------	--	----------------------------	--

6. Pertanyaan

1. Apa masalah yang ditemukan?

Kurang kondisi untuk case jumlah penumpang melebihi kapasitas (anomali)

2. Bagaimana kalian membuktikan masalah tersebut?

Dilihat dari kode if clausenya, terbukti bahwa memang tidak ada kondisi untuk menghandle case penumpang melebihi kapasitas, hanya ada penuh atau tersedia

3. Mengapa perbaikan yang kalian lakukan merupakan solusi teknis yang tepat?

Saya mengambil dari sudut pandang developer dan pemilik usaha.

Developer:

Kalau ada satu celah kecil seperti ini dibiarkan, ini bisa menjadi indikasi bahwa sistem tidak punya lapisan validasi sama sekali soal waktu sebelum data yang lebih parah (misal jumlahPenumpang negatif) juga lolos begitu saja.

Pemilik usaha:

laporan pendapatan ke manajemen bisa jadi lebih tinggi dari kenyataan. Ini bisa menuju ke keputusan bisnis (misal soal ekspansi wahana, evaluasi wahana mana yang laku) berdasarkan angka yang keliru.

7. Bukti Praktikum

Setiap tim menyerahkan:

- Source code terakhir
- 3 test case sebelum perbaikan
- Bukti masalah
- Bagian code yang diubah
- 3 test case setelah perbaikan
- Expected vs Actual
- WHAT-HOW-WHY

Source code akhir:

```
public class KomidiPutar {
    // Atribut
    String kode;
    String nama;
    int kapasitas;
    double hargaTiket;
    int jumlahPenumpang;

    // Constructor
    KomidiPutar(String kode, String nama, int kapasitas,
                double hargaTiket, int jumlahPenumpang) {
        this.kode = kode;
        this.nama = nama;
        this.kapasitas = kapasitas;
        this.hargaTiket = hargaTiket;
        this.jumlahPenumpang = jumlahPenumpang;
    }
}
```

```

// Method menampilkan informasi wahana
void infoWahana() {
    System.out.println("Kode           : " + kode);
    System.out.println("Nama           : " + nama);
    System.out.println("Kapasitas       : " + kapasitas);
    System.out.println("Harga Tiket     : Rp" + hargaTiket);
    System.out.println("Jumlah Penumpang : " + jumlahPenumpang);
}

// Method menghitung pendapatan
double hitungPendapatan() {
    return hargaTiket * jumlahPenumpang;
}

// Method mengecek kapasitas
void cekKapasitas() {
    if (jumlahPenumpang > kapasitas) {
        System.out.println("Status           : MELEBIHI KAPASITAS");
    }
    else if (jumlahPenumpang >= kapasitas) {
        System.out.println("Status           : PENUH");
    } else {
        System.out.println("Status           : TERSEDIA");
    }
}
}

```

```

public class Main {
    public static void main(String[] args) {

        // Array untuk menyimpan 5 object KomidiPutar
        KomidiPutar[] daftarWahana = new KomidiPutar[5];

        // Membuat 5 object dengan data berbeda
        daftarWahana[0] = new KomidiPutar(
            "K01", "Komidi Putar 01", 10, 5000, 10000);

        daftarWahana[1] = new KomidiPutar(
            "K02", "Komidi Putar 02", 12, 5000, 12000);

        daftarWahana[2] = new KomidiPutar(
            "K03", "Komidi Putar 03", 15, 7000, 15000);

        daftarWahana[3] = new KomidiPutar(
            "K04", "Komidi Putar 04", 10, 5000, 10000);

        daftarWahana[4] = new KomidiPutar(
            "K05", "Komidi Putar 05", 20, 7000, 20000);
    }
}

```

```
// Perulangan untuk memproses seluruh object
for (int i = 0; i < daftarWahana.length; i++) {

    System.out.println("=====");

    daftarWahana[i].infoWahana();

    System.out.println("Pendapatan      : Rp"
        + daftarWahana[i].hitungPendapatan());

    daftarWahana[i].cekKapasitas();

}
}
```